

Tema 4: Tipos Avanzados de Datos

Oscar Perpiñán Lamigueiro - David Álvarez

1 Vectores

2 Cadenas de caracteres

3 Estructuras

Vectores en C

Definición

Conjunto de valores numéricos del mismo tipo

Código

```
tipo identificador[tamaño];
```

tipo Tipo de los elementos del vector (int, float, etc.).

identificador Nombre del vector.

longitud Número de elementos (no puede ser una variable).

Ejemplos

```
// Declara un vector llamado miVector compuesto por
// tres elementos de tipo int.
int miVector[3];

// Declara un vector e inicializa todos sus elementos
int miVector[3] = {2, 23, 0};

// Declara un vector e inicializa el primer elemento
// (resto quedan a 0)
int miVector[3] = {2};

// Declara un vector
// La longitud queda determinada por el número de elementos de inicialización
int miVector[] = {2, 23, 24};
```

Elementos de un vector

- Se accede a cada dato con el nombre del vector seguido de un índice entre corchetes.
- El índice representa la posición del elemento dentro del vector.
- La **primera posición** del vector tiene el **subíndice 0**.

```
#include <stdio.h>
```

```
int main(){
```

```
    int miVector[3];
```

```
    miVector[0] = 10;
```

```
    miVector[1] = 2 * miVector[0];
```

```
    miVector[2] = miVector[0] + miVector[1];
```

```
    printf("Posicion 0 = %i\n", miVector[0]);
```

```
    printf("Posicion 1 = %i\n", miVector[1]);
```

```
    printf("Posicion 2 = %i\n", miVector[2]);
```

```
    return 0;
```

```
}
```

Asignación de valores

No se pueden asignar vectores completos, solo dato a dato

```
#include <stdio.h>
int main() {
    int v1[5];
    // Error
    v1 = {1, 2, 3, 4, 5};
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int v1[5] = {1, 2, 3, 4, 5}, v2[5];
    // Error
    v2 = v1;
    return 0;
}
```

Longitud del vector

La dimensión de un vector es un valor constante: **no puede usarse una variable.**

```
# define N 10
int main() {
    // Correcto: el precompilador sustituye N por el valor constante 10
    int miVector[N];

    return 0;
}
```

```
#include <stdio.h>
int main() {
    int n = 10;
    // INCORRECTO, n es una variable
    int miVector[n];

    return 0;
}
```

Acceso a datos de un vector, bucle FOR

```
#include <stdio.h>

int main()
{
    float misdatos[5] = {2.1, 4.9, 0.51, 4.3, 9.01};
    float suma = 0;
    int i;
    // Es común el uso de bucles for para recorrer un vector.
    // Es importante recordar que el primer elemento tiene índice 0.
    for (i = 0; i < 5; i++)
    {
        printf("El elemento %i es %f\n", i, misdatos[i]);
        suma = suma + misdatos[i];
    }
    printf("La temperatura media es: %f\n", suma/5);
    return 0;
}
```


Ejemplo: buscar el número más grande es un vector de 5 datos float.

```
#include <stdio.h>

// Busca e imprime el valor y el índice del número más alto
int main()
{
    float temp[5] = {2.1, 4.9, 0.51, 9.01, 4.3};

}
```

Vectores de Dimensión desconocida

Solución: definir un vector de dimensión elevada y usar sólo los elementos necesarios.

Ejemplo: buscar el y guardar en un vector los número primos del 2 al 300.

```
#include <stdio.h>
int main()
{
    // Definimos un vector de dimension suficiente
    int num_primos[300];
    // Definimos una variable para contar cuantos encontramos
    int contar_primos = 0;

    // Complete el código para calcular los número primos
    // del 2 al 300 y los guarda en ee vector num_primos
    // luego imprima por pantalla los números

    return 0;
}
```

1 Vectores

2 Cadenas de caracteres

3 Estructuras

Cadenas de caracteres en C (*strings*)

Definición

Tipo particular de vector de caracteres (*char*) que acaba con el caracter `'\0'`.

Código

```
char identificador[tamaño];
```

tipo char

identificador Nombre de la cadena.

tamaño Número de elementos de la cadena **incluido el carácter de cierre** (`\0`).

Definición e Inicialización de cadenas

```
// Declara una cadena de 4 caracteres (+1 para el cierre)
```

```
char cadena[5];  
cadena[0] = 'H';  
cadena[1] = 'o';  
cadena[2] = 'l';  
cadena[3] = 'a';  
cadena[4] = '\0';
```

```
// Declara y asigna contenido
```

```
char cadena[5] = "Hola"; // 4 + 1  
char cadena[] = "Hola"; // 4 + 1
```

Esto es un vector, no una cadena

```
char cadena[5] = {'H', 'o', 'l', 'a'};
```

Imprimir elementos de una cadena

- Como en un vector, el índice es la posición del elemento dentro de la cadena.
- La **primera posición** tiene el **subíndice 0**.
- La **última posición** es el carácter nulo \0.

```
int main()
{
    char cadena[] = "Hola buenos dias.";
    int i=0;
    while(cadena[i] != '\0')
    {
        printf("%c ", cadena[i]);
        i++;
    }
    printf("\n"); // Cambiamos de línea
    printf("%s\n", cadena); // Se imprime la cadena completa
    printf("%.8s\n", cadena); // Se imprimen 8 elementos de la cadena completa
    return 0;
}
```

Lectura y escritura de una cadena

- Usamos el especificador %s con printf y scanf.
- En scanf **debemos** especificar el **límite de caracteres** en el especificador de formato.
- **No** ponemos & delante del identificador.

```
#include <stdio.h>

int main()
{
    char texto[8];

    printf("Dime algo: \n");
    // Deja de leer cuando detecta un espacio
    // Imponemos el límite de caracteres
    scanf("%7s", texto);
    printf("Has dicho %s", texto);
    return 0;
}
```

Lectura de una cadena con espacios

- scanf con %s termina de leer cuando recibe un espacio o salto de línea.
- Para leer cadenas con espacios se emplea el identificador %[^\n]
- O usar la función **gets**

```
#include <stdio.h>

int main()
{
    char texto[11];
    printf("Dime algo: \n");
    // Deja de leer cuando detecta un salto de línea
    // o al alcanzar el límite de caracteres
    scanf("%10[^\n]", texto);

    printf("Has dicho %s\n", texto);
    return 0;
}
```


Lectura de una cadena con espacios (2), gets

- scanf con %s termina de leer cuando recibe un espacio o salto de línea.
- Para leer cadenas con espacios se emplea el identificador %[^\n]
- O usar la función **gets**

```
#include <stdio.h>

int main()
{
    char texto[11];
    printf("Dime algo: \n");

    // No se controla el límite de caracteres
    gets(texto);

    printf("Has dicho %s\n", texto);
    return 0;
}
```

Recorrido por los elementos hasta el final de la cadena

- El bucle while es el más indicado, usando el carácter nulo para terminar:

```
#include <stdio.h>

int main()
{
    char cadena[5] = "Hola";
    int i = 0;
    printf("Los caracteres son:\n");
    while (cadena[i] != '\0')
    {
        printf("%c \t", cadena[i]);
        i++;
    }
    return 0;
}
```

Ejemplo: pasar a mayúsculas

```
#include <stdio.h>

// Leer una cadena del teclado
// Pasar a mayúsculas las letras necesarias

int main() {
    char cadena[20];

}
```

Librería string.h

La librería string.h incluye numerosas funciones dedicadas a cadenas de caracteres:

```
#include <string.h>
```

Longitud de una cadena `strlen`

Paso a mayúsculas `strupr`

Copiar cadenas `strcpy`

Concatenar cadenas `strcat`

Comparación de cadenas `strcmp`

Longitud de una cadena :: strlen

- strlen devuelve un entero con el número de caracteres.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char palabra[21];
    int longitud;
    printf("Introduce una palabra: ");
    scanf("%20s", palabra);
    longitud = strlen(palabra);
    printf("Esta palabra tiene %i caracteres\n", longitud);

    return 0;
}
```

Copiar cadenas :: strcpy

Con strcpy tenemos una solución óptima para la **asignación de contenido**.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[50], s2[50];
    strcpy(s1, "Hello World!");
    strcpy(s2, s1);
    printf("%s\n", s2);

    return 0;
}
```

La cadena receptora debe tener espacio suficiente: *los caracteres sobrantes serán eliminados.*

Concatenar cadenas :: strcat

```
#include <stdio.h>
#include <string.h>

int main() {
    char nombre_completo[50];
    char nombre[ ] = "Juana";
    char apellido[ ] = "de Arco";
    // Copiamos por tramos:
    // Primero el nombre
    strcpy(nombre_completo, nombre);
    // A continuacion un espacio
    strcat(nombre_completo, " ");
    // Finalmente el apellido
    strcat(nombre_completo, apellido);
    printf("El nombre completo es: %s.\n", nombre_completo);

    return 0;
}
```

Comparación de cadenas :: strcmp

- Si las dos cadenas son iguales entrega un 0.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char color[] = "negro";
    char respuesta[11];
    do // El bucle se repite mientras
    { // las cadenas *no* coincidan
        printf("Adivina un color: ");
        scanf ("%10s", respuesta);
    } while (strcmp(color, respuesta) != 0);
    printf("¡Correcto!\n");
    return 0;
}
```


Comparación de cadenas :: strcmp

- Si hay diferencias, es positivo si el valor ASCII del primer carácter diferente es mayor en la cadena 1.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[] = "abcdef";
    char s2[] = "abCdef";
    char s3[] = "abcdff";
    int res;
    res = strcmp(s1, s2);
    printf("strcmp(s1, s2) = %i\n", res);
    res = strcmp(s1, s3);
    printf("strcmp(s1, s3) = %i\n", res);

    return 0;
}
```

1 Vectores

2 Cadenas de caracteres

3 Estructuras

Estructuras en C

Permiten almacenar valores de diferentes tipos bajo un mismo identificador.

```
struct identificador
{
    tipo_1 comp_1;
    tipo_2 comp_2;
    ...
    tipo_n comp_n;
};
```

```
typedef struct
{
    tipo_1 comp_1;
    tipo_2 comp_2;
    ...
    tipo_n comp_n;
} identificador;
```

identificador Nombre de la estructura

tipo_n Tipo de datos del componente comp_n.

comp_n Nombre de la variable/vector componente n-ésimo de la estructura.

Ejemplo con struct

```
#include <stdio.h>

struct contacto
{
    char nombre[30];
    int telefono;
    int edad;
    float notas[5];
};

int main()
{
    struct contacto person1;

    return 0;
}
```

```
#include <stdio.h>

typedef struct
{
    char nombre[30];
    int telefono;
    int edad;
    float notas[5];
} contacto;

int main()
{
    contacto person1;

    return 0;
}
```

Inicialización de valores en estructuras

Si no se especifica el identificador de cada componente la asignación se realiza en orden

```
#include <stdio.h>

typedef struct
{
    char nombre[50];
    char apellidos[50];
    int matricula;
} ficha;

int main ()
{
    ficha alumno1 = {"David", "Alvarez Sanchez", 1234};

    return 0;
}
```

Asignación de valores en estructuras

```
typedef struct {  
    int day;  
    int month;  
    int year;  
} date;  
  
int main () {  
    date d1, d2, d3;  
    // Asignacion por componentes  
    d1.day = 31;  
    d1.month = 12;  
    d1.year = 1999;  
  
    // Asignacion por copia  
    d3 = d1;  
    return 0;  
}
```

Asignación de cadenas en estructuras

```
#include <stdio.h>
#include <string.h>

typedef struct {
    char nombre[50];
    char apellidos[50];
    int matricula;
} ficha;

int main ()
{
    ficha alumno1, alumno2, alumno3;
    // Para asignar cadenas usamos strcpy o un código equivalente
    strcpy(alumno1.nombre, "David");
    strcpy(alumno1.apellidos, "Alvarez Sanchez");
    alumno1.matricula = 1234;
    return 0;
}
```

Escritura de componentes de una estructura con scanf

```
#include <stdio.h>

typedef struct
{
    char nombre[50];
    char apellidos[50];
    int matricula;
    float notas[2];
} ficha;
```

// Continúa el código a la derecha ...

```
int main()
{
    ficha alumno;
    printf("Nombre:");
    scanf("%s", alumno.nombre);
    printf("Apellidos:");
    scanf("%49[^\n]", alumno.apellidos);
    printf("Numero de matricula:");
    scanf("%i", &alumno.matricula);
    printf("Nota matematicas:");
    scanf("%f", &alumno.notas[0]);
    printf("Nota fisica:");
    scanf("%f", &alumno.notas[1]);

    return 0;
}
```


Estructuras dentro de estructuras

Una estructura puede contener otras estructuras.

```
#include <stdio.h>

typedef struct
{
    int d, m, a;
} fecha;

typedef struct
{
    char nombre[50];
    char apellidos[50];
    int matricula;
    fecha fNacimiento;
} ficha;
```

```
// ... continuacion del código de la izquierda
int main()
{
    ficha alumno1, alumno2;

    alumno1.fNacimiento.d = 31;
    alumno1.fNacimiento.m = 12;
    alumno1.fNacimiento.a = 1999;

    alumno2.fNacimiento = (fecha){1, 1, 2000};

    return 0;
}
```

Vector de estructuras

Después de definir una estructura, se pueden generar vectores basados en esa estructura.

```
#include <stdio.h>

typedef struct {
    int day;
    int month;
    int year;
} date;

int main() {
    date fechas[3] = { // Vector de 3 fechas
        {1, 1, 1999},
        {31, 12, 2000},
        {15, 5, 1980}
    };
    return 0;
}
```

Vector de estructuras

La asignación de valores sigue las mismas reglas que para vectores de tipos simples (mediante corchetes []).

```
#include <stdio.h>

typedef struct
{
    int day;
    int month;
    int year;
} date;

int main() {
    date fechas[3]; // Vector de 3 fechas
    fechas[0].day = 1;
    fechas[1] = (date) {31, 12, 1999};
    fechas[2] = fechas[1];
    return 0;
}
```